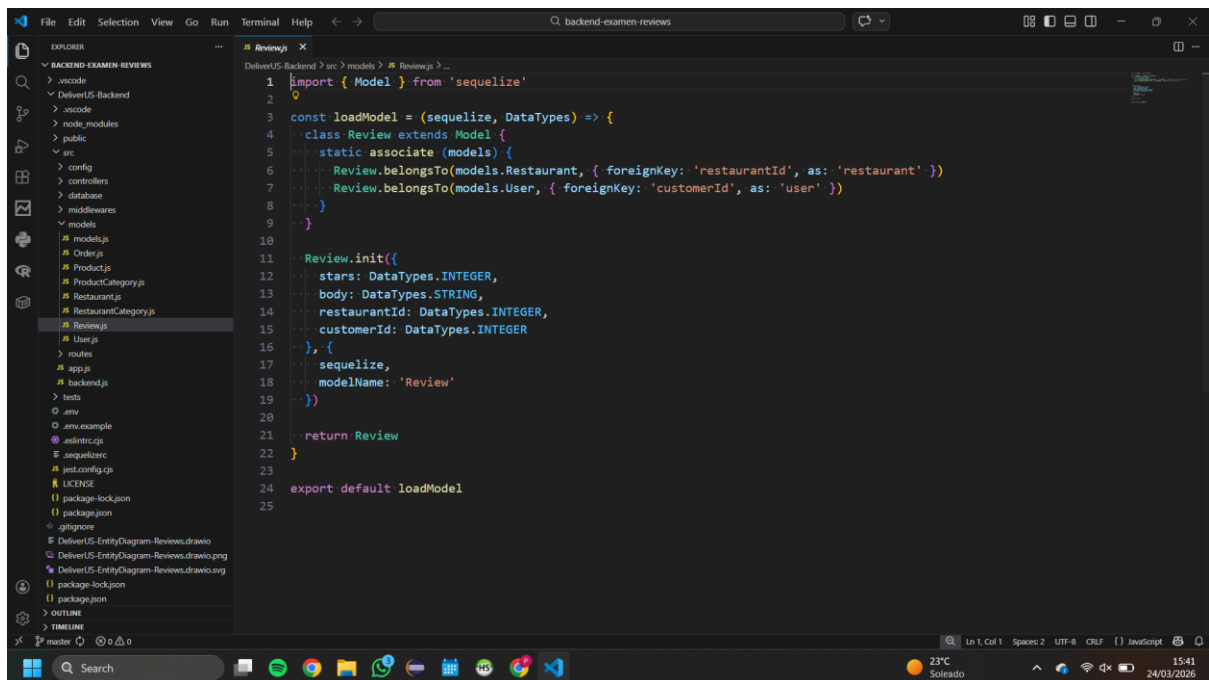
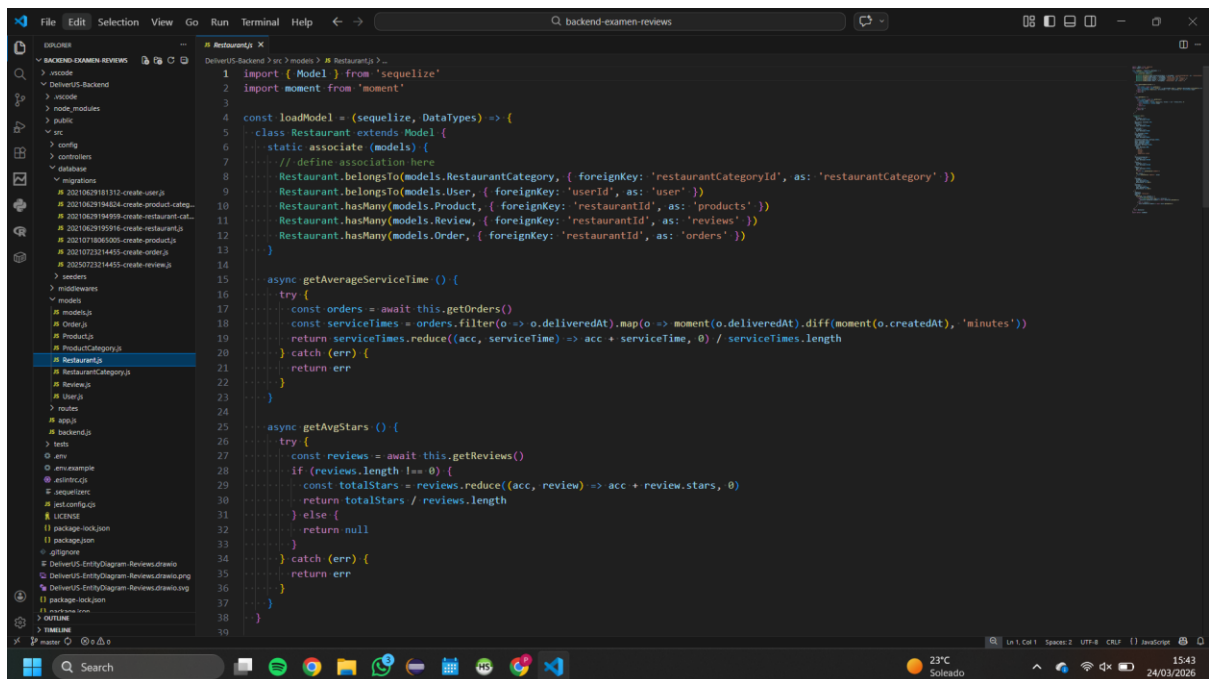


MODELO



The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `Review.js` file. The code defines a `Review` model that extends `Model` from `sequelize`. It includes associations with `Restaurant` and `User` models. The `Review` model is initialized with `sequelize` and `modelName: 'Review'`.

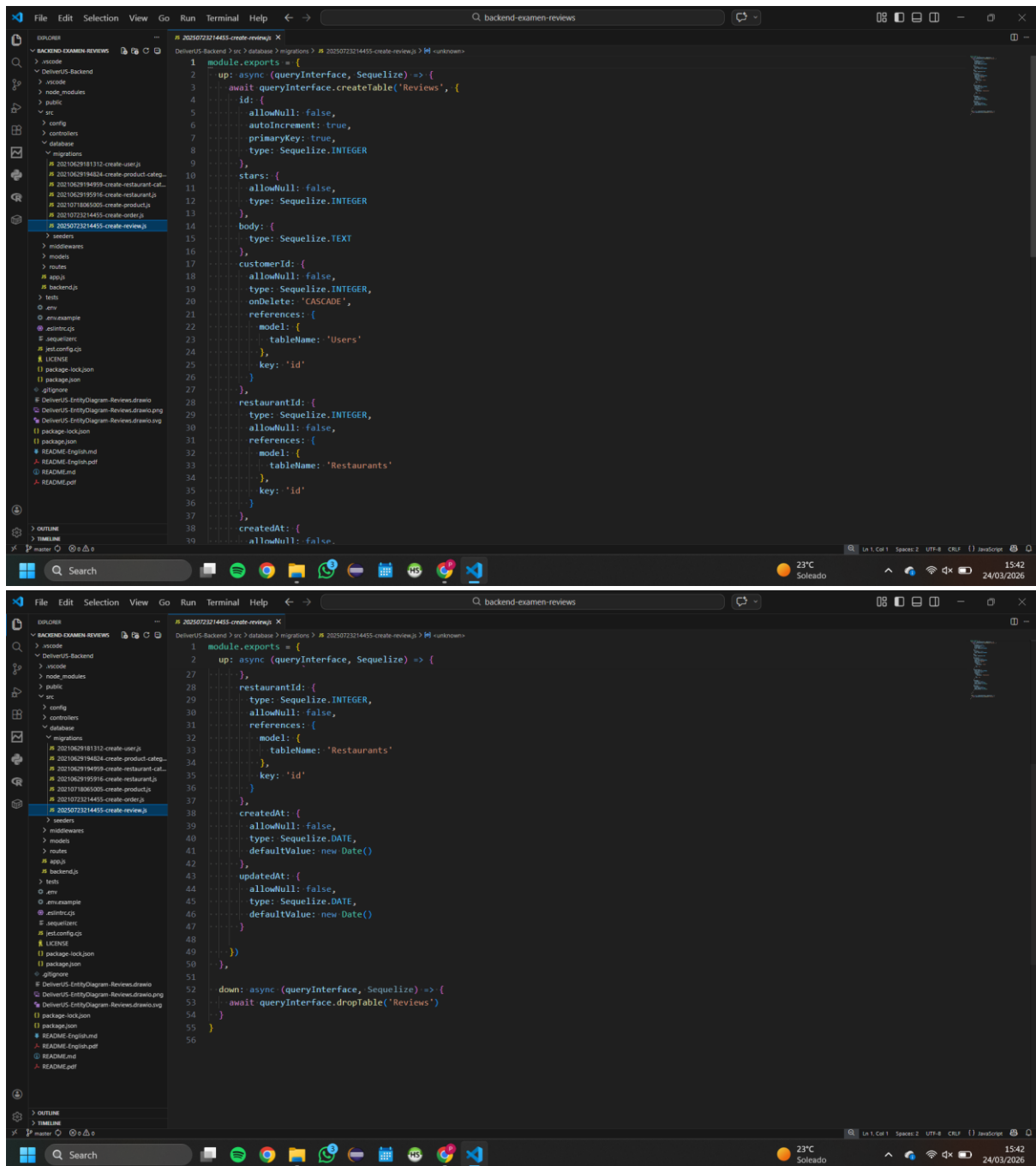
```
1 import { Model } from 'sequelize'
2
3 const loadModel = (sequelize, DataTypes) => {
4   class Review extends Model {
5     static associate (models) {
6       Review.belongsTo(models.Restaurant, { foreignKey: 'restaurantId', as: 'restaurant' })
7       Review.belongsTo(models.User, { foreignKey: 'customerId', as: 'user' })
8     }
9   }
10
11   Review.init({
12     stars: DataTypes.INTEGER,
13     body: DataTypes.STRING,
14     restaurantId: DataTypes.INTEGER,
15     customerId: DataTypes.INTEGER
16   }, {
17     sequelize,
18     modelName: 'Review'
19   })
20
21   return Review
22 }
23
24 export default loadModel
25
```



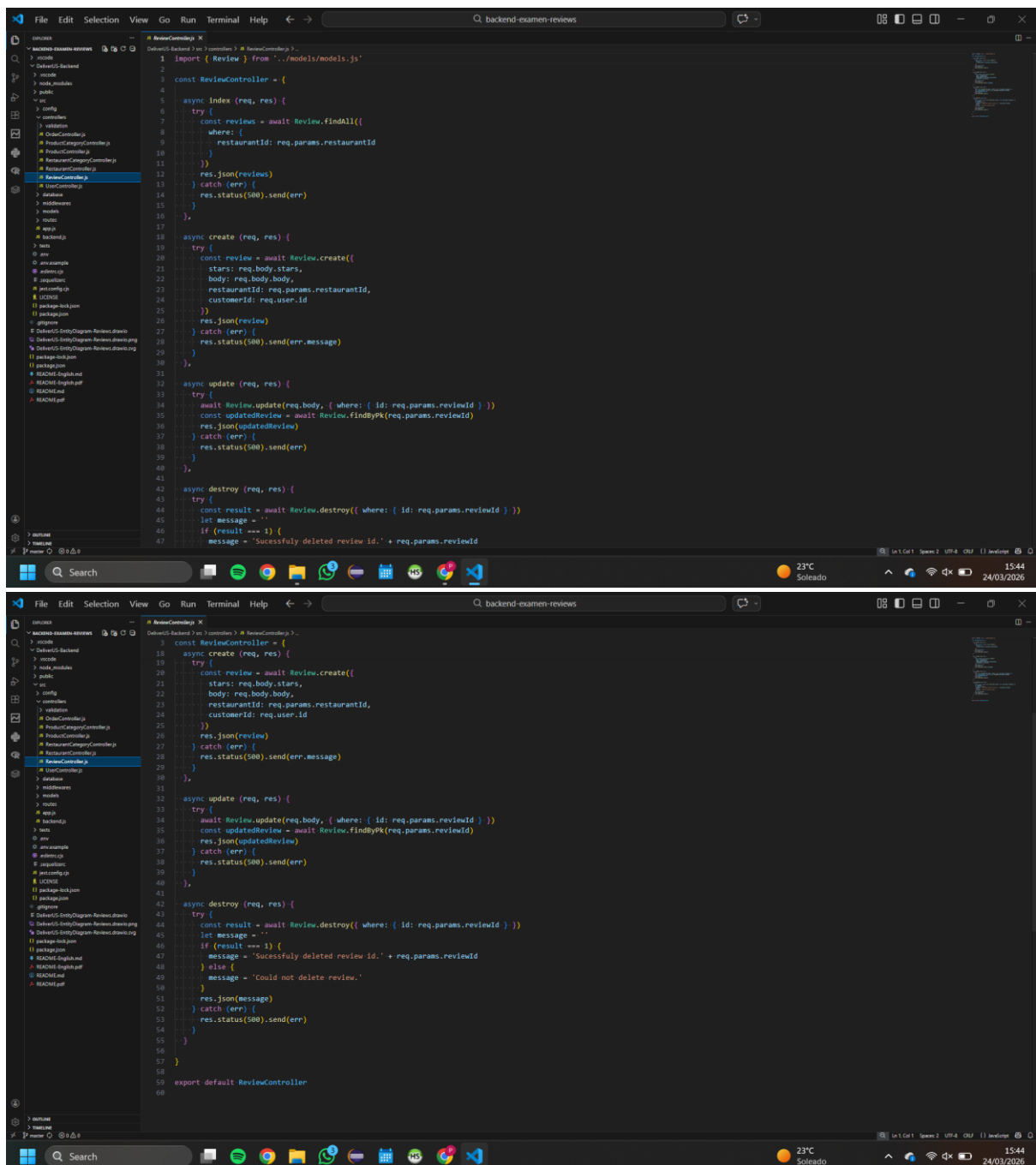
The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `Restaurant.js` file. The code defines a `Restaurant` model that extends `Model` from `sequelize`. It includes associations with `RestaurantCategory`, `User`, `Product`, `Review`, and `Order` models. The `Restaurant` model is initialized with `sequelize` and `modelName: 'Restaurant'`. It also includes two asynchronous methods: `getAverageServiceTime` and `getAvgStars`.

```
1 import { Model } from 'sequelize'
2 import moment from 'moment'
3
4 const loadModel = (sequelize, DataTypes) => {
5   class Restaurant extends Model {
6     static associate (models) {
7       // define association here
8       Restaurant.belongsTo(models.RestaurantCategory, { foreignKey: 'restaurantCategoryId', as: 'restaurantCategory' })
9       Restaurant.belongsTo(models.User, { foreignKey: 'userId', as: 'user' })
10      Restaurant.hasMany(models.Product, { foreignKey: 'restaurantId', as: 'products' })
11      Restaurant.hasMany(models.Review, { foreignKey: 'restaurantId', as: 'reviews' })
12      Restaurant.hasMany(models.Order, { foreignKey: 'restaurantId', as: 'orders' })
13    }
14
15    async getAverageServiceTime () {
16      try {
17        const orders = await this.getOrders()
18        const serviceTimes = orders.filter(o => o.deliveredAt).map(o => moment(o.deliveredAt).diff(moment(o.createdAt), 'minutes'))
19        return serviceTimes.reduce((acc, serviceTime) => acc + serviceTime, 0) / serviceTimes.length
20      } catch (err) {
21        return err
22      }
23    }
24
25    async getAvgStars () {
26      try {
27        const reviews = await this.getReviews()
28        if (reviews.length !== 0) {
29          const totalStars = reviews.reduce((acc, review) => acc + review.stars, 0)
30          return totalStars / reviews.length
31        } else {
32          return null
33        }
34      } catch (err) {
35        return err
36      }
37    }
38  }
39
40   return Restaurant
41 }
42
43 export default loadModel
44
```

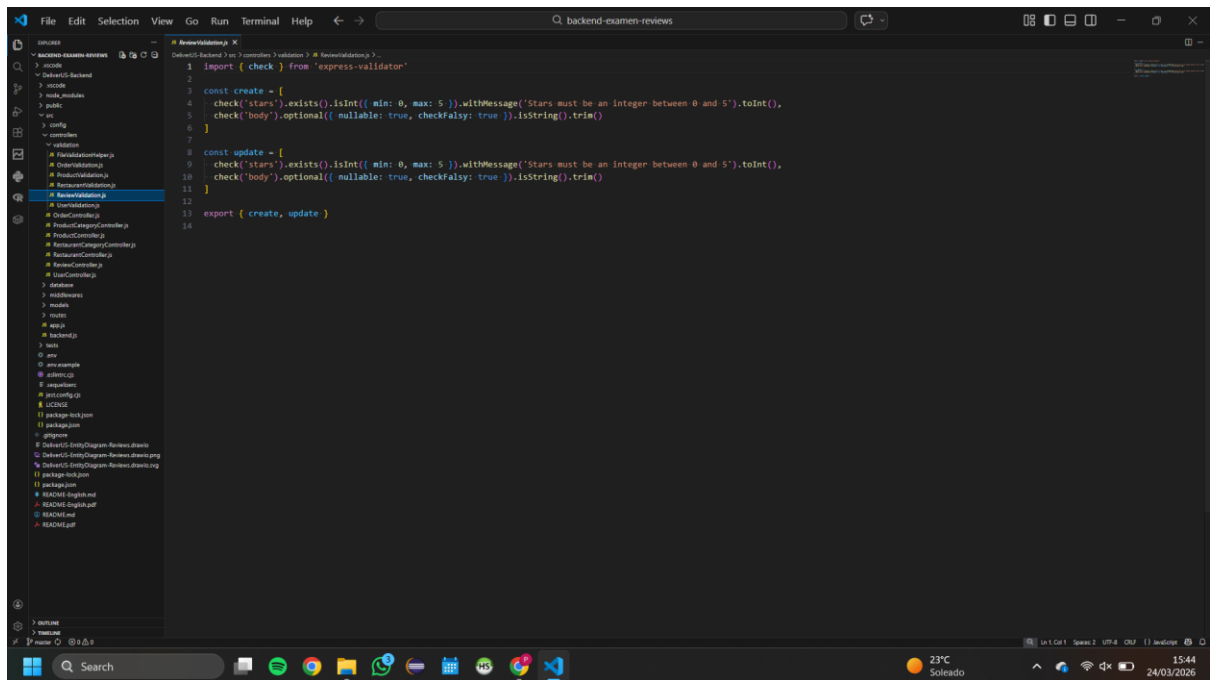
MIGRATION



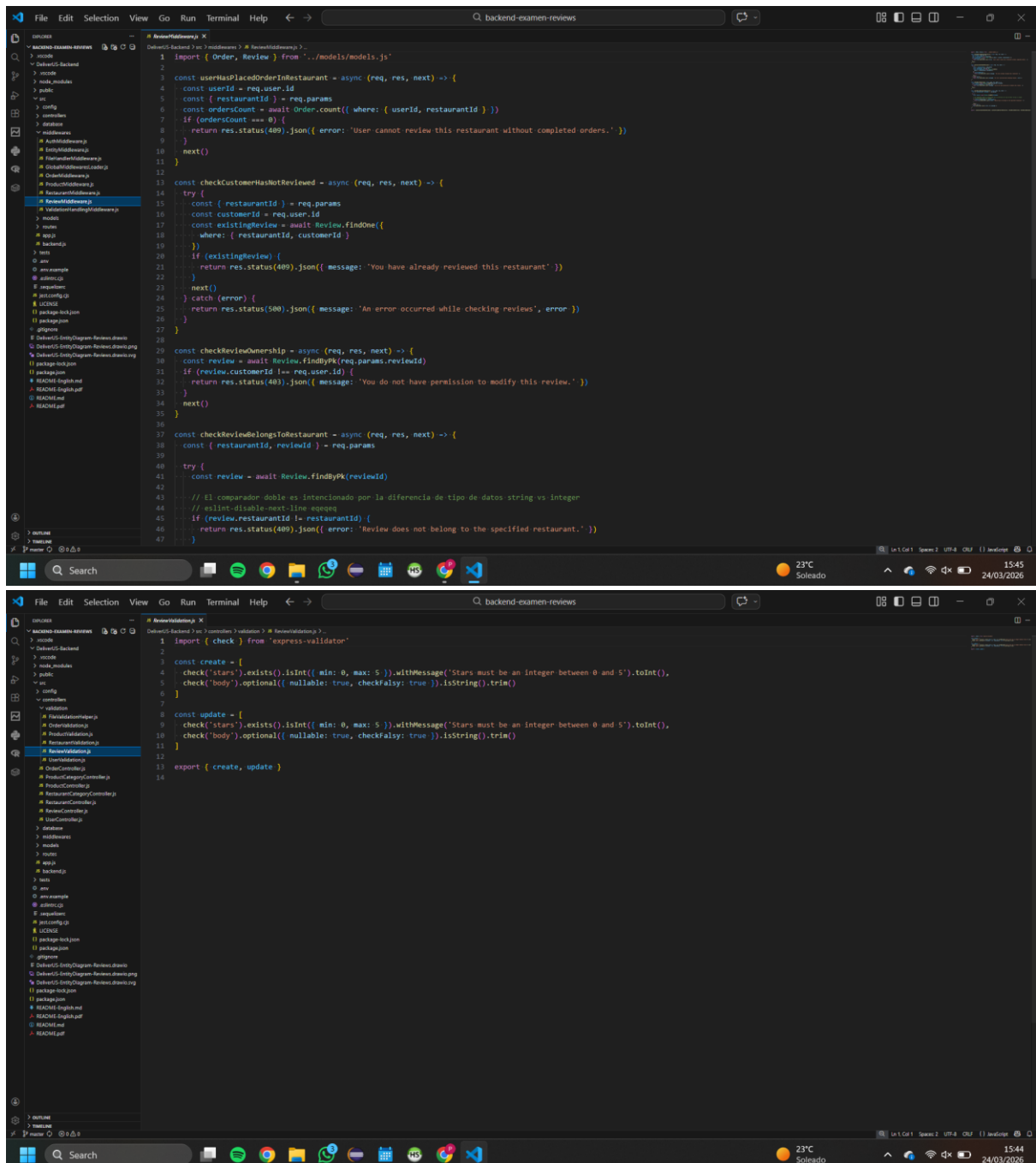
CONTROLLER



VALIDATION



MIDDLEWARES



ROUTES

